

(20 pts) Questão 1: O motor de um gerador de energia tem sua velocidade controlada por um sistema de injeção de combustível. A injeção de combustível é controlada por sinal PWM e deve manter a tensão de saída em 220V. Um conversor AD fornece diretamente o valor da tensão, ou seja, AD = 220 representa 220V. Tipicamente para 220V o sinal PWM é ajustado em 50% porém deve aumentar 1% se a tensão abaixar 1V e diminuir 1% se a tensão subir 1V. Implemente um **programa** para esse sistema.

(25 pts) Questão 2: Um medidor de velocidade e direção possui 3 dígitos para indicar a velocidade e 4



símbolos (N,S,E,W) que indicam a direção. Os símbolos N,S,E,W são controlados pelos bits 0,1,2 e 3 da PORTA. A velocidade é apresentada utilizando 3 displays de 7 segmentos e a biblioteca `ssd.h`. Implemente uma **biblioteca** com as funções `displayInit()`, `displayVelocidade()` e `displayDirecao()` para ser utilizada neste dispositivo. A função `ssdUpdate()` será chamada pelo programa principal e não pela biblioteca.

(25 pts) Questão 3: Um laboratório de física possui um dispositivo que mede o tempo de queda de uma esfera metálica e mostra em um display de LCD o tempo em milisegundos da queda entre dois pontos. Após ativado por um botão (bit 0 PORTA) o contador é acionado por um sensor (bit 1 PORTA) e parado por outro sensor (bit 2 PORTA). O tempo é controlado por um circuito de Timer que controla o loop. Implemente um programa para esse dispositivo. Podem ser utilizadas as bibliotecas `pic18f4520.h` e `config.h`.

(30 pts) Questão 4: Um sistema de aquecimento possui um sensor de temperatura com resposta linear ligado a um conversor ADC de um microcontrolador, quando a temperatura está acima de 200°C o aquecedor deve desligar (bit 5 porta C), quando está abaixo de 180° o aquecedor deve ligar. O conversor ADC fornece valores de 0 a 1023 que correspondem a valores de temperatura de 0 a 300°C. Este sistema possui um display de 7 segmentos que mostra a temperatura com 4 dígitos. Implemente o **programa** utilizando a interrupção do Timer para controlar a atualização do display a cada 1ms.

```
char flag;
//Esta interrupção é chamada a cada 1ms.
void Int(void) __interrupt() {
    if (BitTst(INTCON, 2)) { //Verifica se ocorreu overflow de TMR0
        timerReset(1000); //Ajusta o contador do TMR0
        BitClr(INTCON, 2); //Desliga o Flag de everflow do TMR0
        flag = 1;
    }
}
```

Obs: Uma função `InitInterrupt()` ajusta todos os parâmetros da interrupção.

```
//adc.h
void adcInit(void);
int adcRead(void);
```

```
//ldc.h
void ldcInit(void);
void lcdChar(char letra);
void lcdCommand(char val);
```

```
//pwm.h
void pwmInit(void);
void pwmFrequency(unsigned int freq);
void pwmSet1(unsigned char por cento);
```

```
//config.h
Necessário para configurar o hardware
Possui comandos: #pragma ...
```

```
//pic18f4520.h
Define TRISA, PORTA,... bem como:
BitSet(arg,bit) BitClr(arg,bit)
BitFlp(arg,bit) BitTst(arg,bit)
```

```
//ssd.h
void ssdInit(void);
void ssdUpdate(void);
void ssdDigit(int val, int pos);
```

```
//timer.h
void timerInit(void);
void timerReset(unsigned int tempo);
void timerWait(void);
```

```
//Observação: TRIS = 0 → Saída
//Em ASCII caracter '0' = 48
```